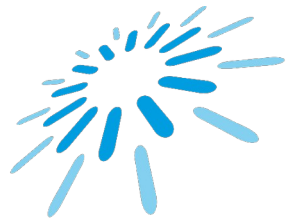




# Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 06



TH Aschaffenburg  
university of applied sciences

|                     | Monday      | Tuesday | Wednesday                                     | Thursday | Friday                       |
|---------------------|-------------|---------|-----------------------------------------------|----------|------------------------------|
| 08:00<br>-<br>09:30 |             |         |                                               |          | Vorlesung<br>Software Design |
| 09:45<br>-<br>11:15 |             |         |                                               |          | Übung<br>Software Design     |
| 11:30<br>-<br>13:00 |             |         | Lecture<br>Software Design<br>International   |          |                              |
|                     | Lunch Break |         |                                               |          |                              |
| 14:00<br>-<br>15:30 |             |         | Exercises<br>Software Design<br>International |          |                              |
| 15:45<br>-<br>17:15 |             |         |                                               |          |                              |
| 17:30<br>-<br>19:00 |             |         |                                               |          |                              |

**What does GRASP stand for?**

**What does KISS stand for?**

## Question

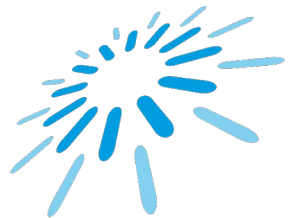
**Systems accessing other systems:  
What kind of interfaces do they use?**



# Web Interfaces 101

## HTTP Protocol

Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg  
university of applied sciences

# Web Interfaces 101



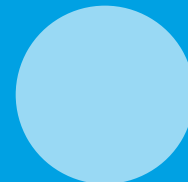
**Recap: HTML, CSS, JS**

**HTTP Protocol**

**HTTP Content – Mime Types and Headers**

**Verbs and URIs**

**Cache**



**What is HTML?**



**What is CSS?**

**What is Javascript?**

# Demo

---

## A simple HTML Page

# HTML, CSS, Javascript

- HTML – Hypertext Markup Language:  
Markup language designed for display in web browsers
- CSS – Cascading Style Sheets:  
Styling language for content displayed with a markup language (such as HTML)
- Javascript:  
Scripting language, evaluated by browser. Allows for dynamic behavior of web pages.
  - Note1: Javascript can also be executed outside of the browser using tools such as nodejs or embedded browsers, such as electron.
  - Note2: **Java**script has nothing to do with Java. The choice of name was a marketing hoax.

# Web Interfaces 101



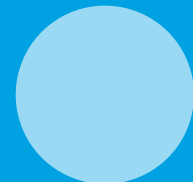
**Recap: HTML, CSS, JS**

**HTTP Protocol**

**HTTP Content – Mime Types and Headers**

**Verbs and URIs**

**Cache**



**What is the TCP Protocol?**

**What is the HTTP Protocol?**

## Question

**What are the main parts of an HTTP message?**



# HTTP – Hypertext Transport Protocol

- Text-based protocol for communication between a client and server
  - Sends plain text via TCP
- Request – Response Protocol: Client sends request, server answers with response
  - Synchronous: Client always expects a response!
- Three parts:
  - Request / Response line – The first line:  
Specifies type of request/response and data required to match it
  - Headers:  
Metadata required to understand and handle the body
  - Body:  
The actual content

# HTTP Request Format

- Request Line: `GET /index.html HTTP/1.0`
- Header Lines (header names are not case-sensitive!):  
`Host: google.com`  
`Date: BBBBBBBB`  
`User-Agent: Mozilla/5.0 (en)`  
`Accept-Language: en-us`
- (*Optional*) Message Body: Message content

# Demo

---

## HTTP Server from Scratch!

***Disclaimer: Don't do this at your Job!***

# HTTP Response Format

- Response Line: HTTP/1.0 200 OK
- Header Lines (header names are not case-sensitive!):  
Host: google.com  
Date: BBBBBBBB  
User-Agent: Mozilla/5.0 (en)  
Accept-Language: en-us
- (*Optional*) Message Body: Message content

# Web Engineering 101



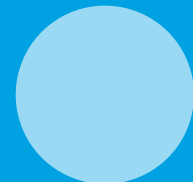
**Recap: HTML, CSS, JS**

**HTTP Protocol**

**HTTP Content – Mime Types and Headers**

**Verbs and URIs**

**Cache**



## Question

**How does the browser know if a resource is an image, an html page, a pdf, or something else?**

# Mime Types

- Type specifications that tell the browser the type of the body's content
- Are specified in headers
  - **Content-Type** header specifies mime type of content
- Browser handles content according to its mime type
- Examples
  - `text/plain` – Plain text, display entire body as plain text
  - `text/html` – HTML Document for rendering
  - `image/png` – PNG Image
  - `audio/mpeg` – MP3 audio file
  - ....

# Demo

—

## Serving Content



# Question

## **Scenario:**

We are serving an audio/mpeg resource. However, we don't want to play it in the browser. Instead, we want the user to immediately download it.

**How do we do this?**

## Question

### **Scenario:**

We are serving an audio/mpeg resource. However, we don't want to play it in the browser. Instead, we want the user to immediately download it.

**How do we do this?**

**Use Mime Type:** application/octet-stream

# Important Headers regarding Mime Types and Content

- Content-Type: Type of Content in this message's body
- Accept: List of content type that this client/server accepts
  - In Request read as: Please respond with this content type
  - In Response read as: Please send your requests with this content type
- Charset: Character encoding of text body
  - Most common example: Charset: UTF-8
- Content-Length: Length of content in bytes
- ...

# Web Engineering 101

**Recap: HTML, CSS, JS**

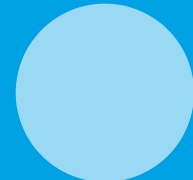
**HTTP Protocol**

**HTTP Content – Mime Types and Headers**



**Verbs and URIs**

**Cache**



## Question

**Our server wants to serve multiple resources and users should be able to perform different types of actions.**

**How do we differentiate between the different requests?**

# Remember: HTTP Request Format

- Request Line: GET /index.html HTTP/1.0

## Method

Specified using Verb

## Resource

Specified using URL – Uniform Resource Locator

Protocol Version

Note URL is a type of URI – Uniform Resource Identifier

# Resources and URLs

- A resource specifies a “thing”, we want to access, examples:
  - List of employees
  - An Image
  - A file
  - A calendar
- We address resources using a directory-like tree-structure, called URL
  - Example: `/companies/awesome-gamestudio/employees`

# URL Parameters

- When accessing dynamically generated resources, we might want to add some parameters to specify the resource further:
  - **Example 1:** The company for which to get the employees
    - URL: `/companies/awesome-gamestudio/employees`
    - Referred to as “**Path parameter**”
    - The company name is parameter as part of the path. Evaluated by server using some application logic (E.g., lookup company name in a database)
  - **Example 2:** Filter for certain employee names
    - URL: `/companies/awesome-gamestudio/employees?firstname=John`
    - Referred to as “**Query parameter**”, can have 0, 1, or more parameters. Query parameters are always optional!
    - URL with query parameter syntax:  
`<PATH>?<PARAMETER_NAME>=<VALUE>&<ANOTHER_PARAMETER_NAME>=<VALUE> ...`



# Demo

---

## Parsing a Query Parameter

# HTTP Methods

- Specified using “*Verbs*”, tell the server type of expected action on the resource
- **GET**: Retrieve the resource
  - GET requests do not contain a body!
- **POST**: Add/modify a resource
  - Body contains information necessary to perform the operation.
- **PUT**: Modify an existing resource
  - Body contains the exact modification.
- **DELETE**: Delete a resource
  - DELETE requests do not contain a body!
- ...

## Question

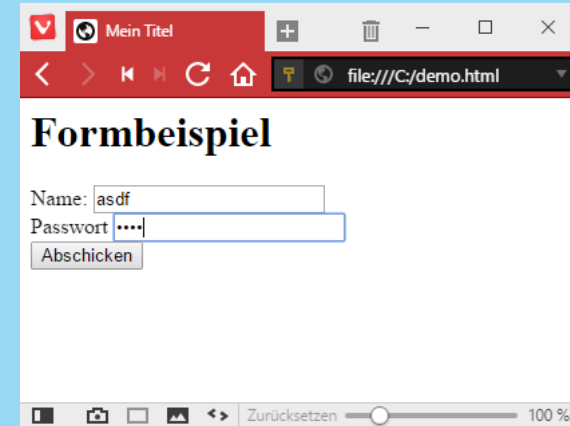
**How do we send requests with content to the server?**

# Sending HTTP Requests from the Browser

- **GET:** Use a link (or HTML Form)
  - `<a href="/myresource">Click here</a>` sends a GET-Request to the resource and displays the response in the browser
- **POST:** Use HTML Form or Javascript
- **PUT:** Use Javascript
- **DELETE:** Use Javascript
- ...

# HTML Forms

```
...  
<h1>Formbeispiel</h1>  
<form method="post" action="demo2" charset="UTF-8">  
  Name: <input name="name" type="text"/><br/>  
  Passwort <input name="password" type="password"/><br/>  
  <input type="submit" value="Abschicken"/>  
</form>  
...
```



- Uses the method specified in “method” (Either GET or POST)
- Sends to the resource URL specified in “action”
- On GET request: Input data is sent as query parameters, with “name” being the parameter name
- On POST request: Input data is sent in body using Mime types `application/x-www-form-urlencoded` or `multipart/form-data`

# HTTP Requests using JavaScript

- Modern: Use fetch-API provided by all modern browsers.
- Example:

```
const response = await fetch("/myresource", {  
  method: "POST",  
  headers: {  
    "Content-Type": "text/plain",  
    "Accept": "text/plain"  
  },  
  body: "Hello World!"  
});
```

# HTTP Response Status Codes (1/2)

Specified using 3-digit numbers. Tell client how request was received and processed

- **100-199**: Informational responses
- **200-200**: Successful responses:
  - 200: Ok – Generic Ok response, most common
  - 201: Created – A new resource was successfully created
  - 204: No content – Request was processed successfully, but no content response will be provided, since none was asked for
- **300-399**: Redirection messages:
  - 301: Moved permanently – Resource exists but is now in new location. New location usually specified in Location header.
  - 302: Moved temporarily – Resource exists but is currently in another location.
  - 304: Not modified – We will get to this one later ;)

# HTTP Response Status Codes (2/2)

- **400-499:** Client error responses
  - 400: Bad request – Generic bad request response
  - 401: Unauthorized – Client should log in first
  - 403: Forbidden – Client does not have permissions
  - 404: Not found
- **500-599:** Server error responses
  - 500: Internal Server Error – Something went wrong in the server
  - 502: Bad Gateway – Proxy or Gateway server is down, response cannot be passed on
  - 503: Service Unavailable – Server currently down
- Full List: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>



# Demo

---

## HTML Form

# Web Engineering 101

**Recap: HTML, CSS, JS**

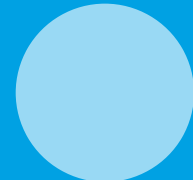
**HTTP Protocol**

**HTTP Content – Mime Types and Headers**

**Verbs and URIs**



**Cache**



# Question

## **Scenario:**

We have a website with lots of large pictures.

**How do we stop the browser from downloading all pictures over and over again when a user revisits our site?**

# Cache-Control

- Header that tells the browser to cache a response (or not) and reuse it instead of sending the request again
- E.g.:
  - `Cache-Control: max-age=3600` – Keep and reuse the response for up to 3600 seconds
  - `Cache-Control: no-store, max-age=0` – Never keep the response, always get it again
- **Warning:** There are a lot of different headers and not all browsers respect all of them. As a result you will often see multiple different headers or multiple values being used.
  - `Cache-Control: no-store, max-age=0` – Multiple cache control values in case a browser does not respect one of the two
  - `Pragma: no-cache` – Ancient Header used to disable cache in old browsers

# Other Caching Headers

- Etag
  - Server returns a hash or version of the resource.
  - This is then sent along with any future requests using If-None-Match header.
  - Server only returns response content if Etag has changed, otherwise it responds with empty response and status code 304 - Not modified
- Last-Modified
  - Server returns point in time when resource was last modified
  - This is then sent along with any future requests using If-Modified-Since header.
  - Server only returns response content if content has changed, otherwise it responds with empty response and status code 304 - Not modified

# Question

## **Scenario:**

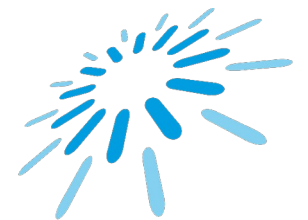
You have updated some Javascript code, restarted your development server, but the new code is not executed by the browser.

**Why is that? How can you fix it?**



# Until next time!

Software Architecture and Design Patterns  
Prof. Dr. Jóakim von Kistowski



**TH Aschaffenburg**  
university of applied sciences